A More Behavioral Study of Greater-Than in GPT-2

In this section, we discuss the results of additional behavioral studies of GPT-2's behavior.

GPT-2 also predicts valid XX Although our previous experiments show that GPT-2 predicts a valid two-digit end year YY, we also verify that GPT-2 predicts a valid continuation when the output prefix XX is not provided. In this scenario, GPT-2 should either predict either a two-digit century token whose value is $\geq$ XX, or an entire 4-digit year token that is $\geq$ XXYY. We test this using the same dataset as in previous experiments, with the final XX removed. We find that on average, considering the top-100 tokens (95% of probability mass), 94% of their probability mass (89% of the total) is assigned to valid continuations. So, GPT-2 generally predicts valid XX.

GPT-2's top predictions are valid As suggested by the high probability difference achieved by GPT-2, its top YY predictions for an input from our dataset are quite good. 100% of the top-1 continuations, and 98.6% of top-5 continuations were correct.

Default Behavior in GPT-2 GPT-2 predicts the greater-than in greater-than scenarios, but also in scenarios where less-than would be appropriate (see Section 5). For this reason, we also tested how GPT-2 behaves in a scenario where neither greater-than nor less-than in required, in order to a elicit a sort of default behavior.

To test this, we created sequences of 4-digit numbers like "XXY1, XXY2,..., XXYN, XX". We chose XX, Y1,. . . ,YN randomly for each sequence, respecting tokenization. Sequences were generally not monotone, so any 2-digit continuation YY could have been valid. We found that GPT-2's behavior depends on the sequence length N. At low N, GPT-2 produces mostly numbers $>$ YN; the proportion of continuations $<$ YN increases smoothly until 50% by $N = 20$. So even in the context of random sequences, GPT-2 often generates increasing numbers.

Beyond simple behavioral study, we conducted a circuits analysis on this task, and found that our greater-than circuit also underlies this behavior. This provides another example of GPT-2's ability to identify greater-than situations is flawed. However, it also demonstrates again that whenever we see this greater-than behavior, our circuit is responsible. An open question worth answering is why our circuit activates in such incorrect scenarios, and if this can be mitigated.

B The Full Year-Span Prediction Circuit

Now, we describe the rest of the year-span prediction circuit. This is actually not very large, as we already know most of the important components. All that remains is to understand how the input to the attention heads is crafted.

We can investigate this via iterative path-patching again: we will look for nodes that influence the attention heads. This can be done in three ways: via queries, keys, and values. The queries and keys jointly determine what the attention heads attend to. In theory, attention patterns should be relatively constant across examples: in all cases, attention heads should attend to the YY position. If this is the case, we should be able to ignore the queries and keys, and focus only the value.

In practice, attention patterns are not exactly constant as YY changes. While broad trends are similar, the intensity of the attention to the YY position varies (though not linearly with YY, as might make sense: see Figure 11). At YY=01, attention to the YY position is rather low, meaning that model greater-than behavior is less pronounced when patched. Thus, the queries and keys are somewhat important: patching them with bad data reduces performance by 15% with respect to our partial circuit. The influences on these heads via the keys are similar to those on the values, discussed in the next paragraph. The influences via the queries are distinct, but we will set these aside, and focus on the value vectors.

The most important influences on these heads are the influences on their values at the YY position. The values are combined to form each attention head's output: patching the values with 01-input entirely disrupts circuit performance, unlike patching its keys or queries.

To find influences on these values, we iteratively path patch potential components that might communicate with our attention heads via their values at the YY position. We find (Figure 12) that these

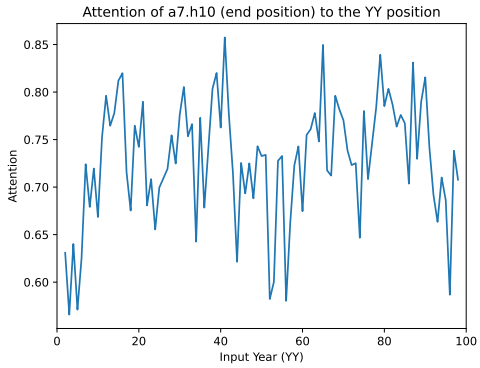

Figure 11: Attention from a7.h10 (end position) to the YY position, by input year YY

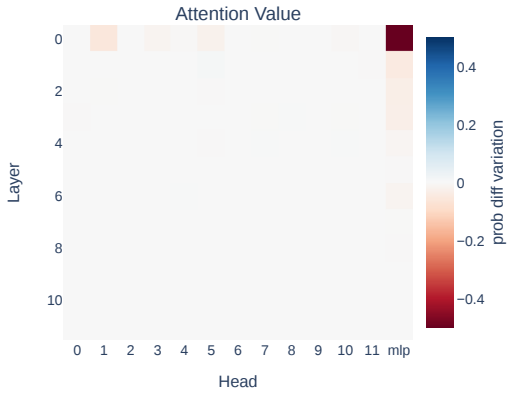

Figure 12: Iterative path patching results through attention heads' value vectors

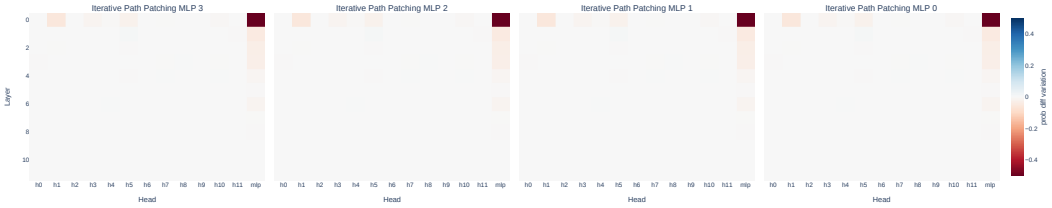

Figure 13: Iterative path patching results for MLPs 0-3

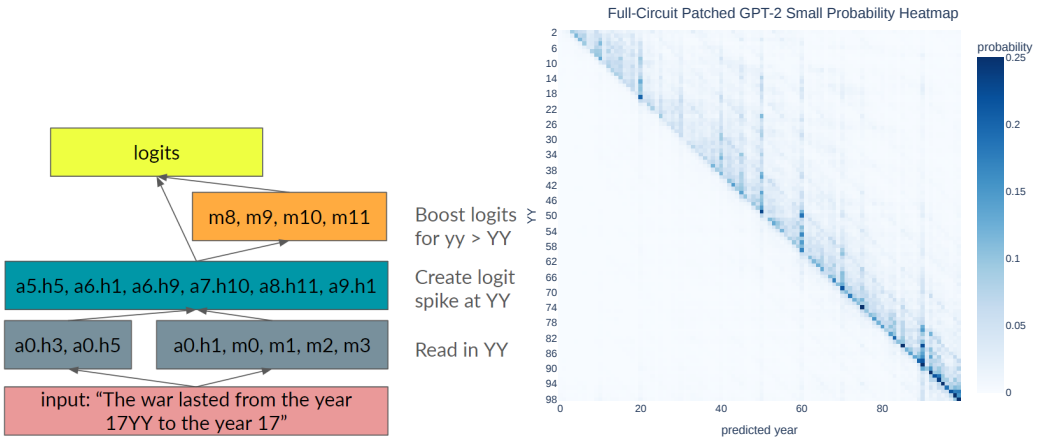

Figure 14: **Left**: Full circuit diagram. Note that all components in the boxes containing MLPs are interconnected; the components in boxes containing only attention heads are not. The components in the gray boxes in gray jointly contribute to the keys and values of the components in the teal box. **Right**: Probability heatmap for the patched full circuit.

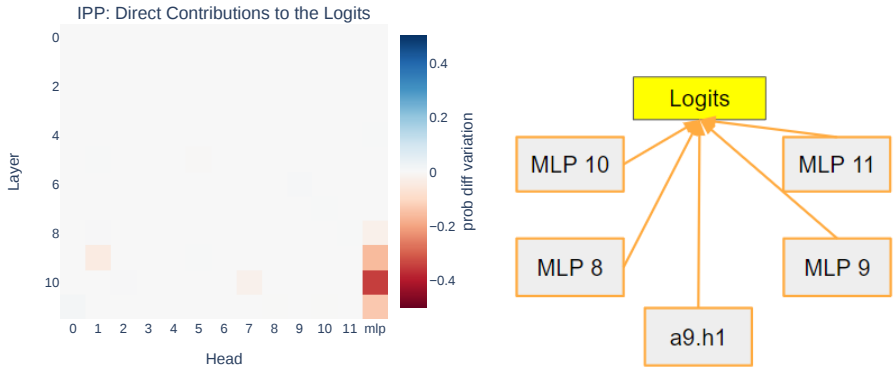

Figure 15: Path Patching Step 1: Logits

are mostly MLPs 0-3, as well as a0.h5, a0.h3, and a0.h1. We delve again into this group of MLPs (Figure 13), and see that each MLP relies on the MLPs before it and a0.h1; MLP 2 is the exception, as it does not rely on MLP 1. We know that a0.h5 can only rely on the token embeddings; there is nothing else before it in the residual stream! We attempt to find what MLP 0 depends on, but it does not rely on any of the attention heads prior to it; this indicates that it depends primarily on the token embeddings, not shown in these iterative path patching diagrams.

We have now developed a hypothesis regarding a full circuit, pictured in Figure 14. We evaluate it as before, keeping in mind that the keys and values of the attention heads are patched with the same input components as found earlier; the queries, not the focus of this section, receive all good inputs. The circuit achieves a probability difference of 71.5% (98.3% of what we achieved earlier), and a cutoff sharpness of 10.5% (again sharper than pre-patching). The qualitative results are in Figure 14.

C Circuit Finding, Step by Step

In this section, we explain the circuit finding procedure step by step, with additional diagrams to aid comprehension. We start, as indicated previously, by patching direct connections to the logits (Figure 15). This reveals connections to the logits from MLPs 8-11, as well as a9.h1. We continue with the next furthest downstream MLP, MLP 11, and see which nodes influence the circuit via it. Note that the only path through which a node C can influence the circuit via MLP 11 is (C, MLP 11,